

C Programming: Lecture II

FYD401



Credits

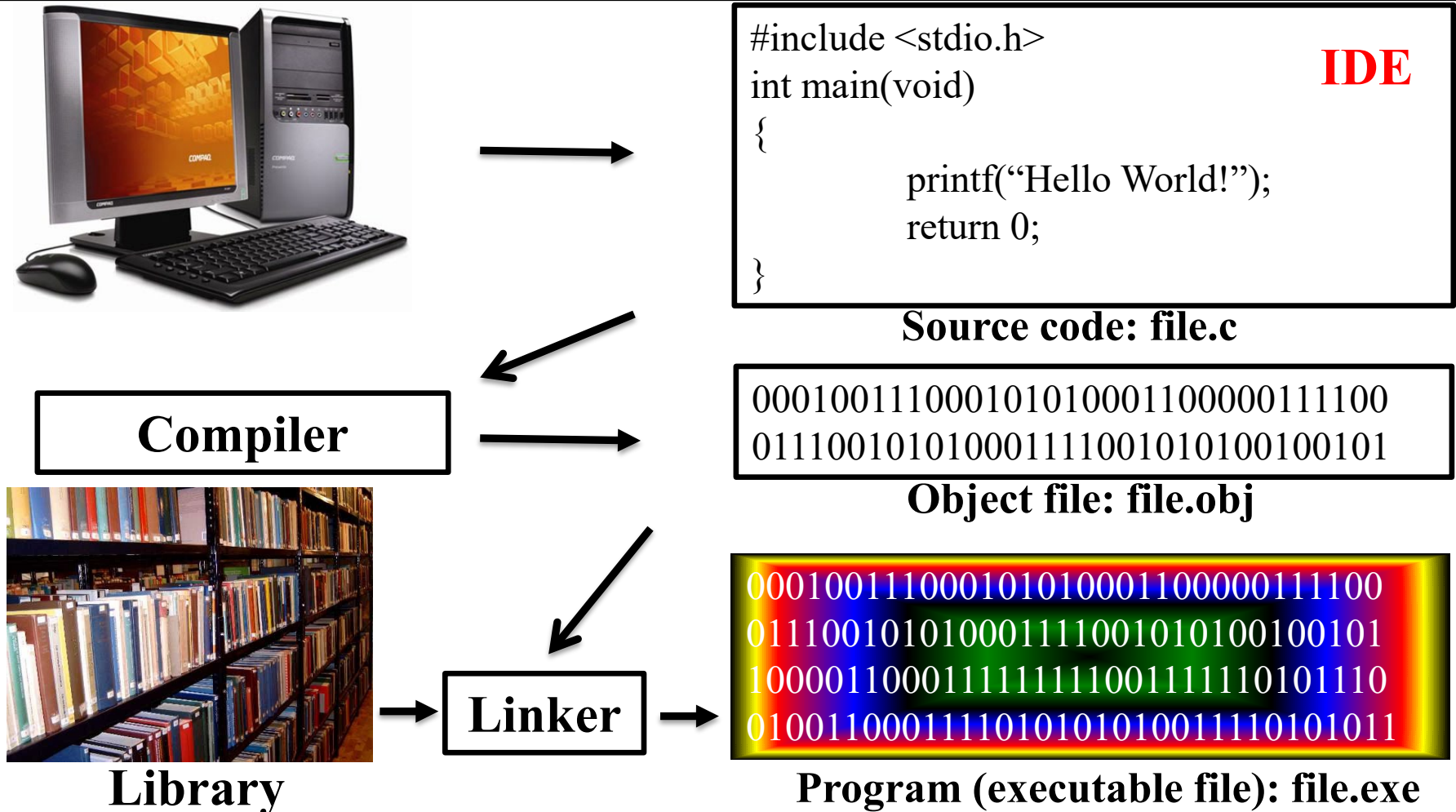
- The slides for the course were prepared by Andreas Heinz (Chalmers University of Technology)
 - Minor modifications were made
-

Contents

- **Vocabulary**
 - **Algorithms**
 - **Data types**
 - **Preprocessor**
-

Some vocabulary ...

How to Create a Program?



Vocabulary

- **Source code**

- Collection of instructions written by a programmer in a programming language

- **Compiler**

- Translates a source code written in a programming language into a target language that can be executed by the computer

- **Linker**

- Combines the objects created by a compiler and the necessary libraries into an executable, library or object

- **Executable (in Windows: filename.exe)**

- Computer performs the tasks according to the encoded instructions
-

Vocabulary

- **Typical source program in C**

- **Preprocessor directives**

- **Lines starts with #, instructions followed before compilations (e.g., include a header file, define a macro, ...)**

- **Functions**

- **Variables**

- **Statements and Expressions**

- **Comments**

```
#include <stdio.h> // Preprocessor directive
int main (void) // main function
{
    int a=1; // variable a
    printf ("a = %d",a); // function printf
    getchar(); // function
    return 0;
}
```

Vocabulary

- **Header files**

- C function declarations and macro definitions that can be shared between several source files
- Provided with the IDE or by the programmer

- **The compiler knows how to call the functions via the header files ...**

- **C standard libraries**

- Set of C built-in functions
-

What is an algorithm?

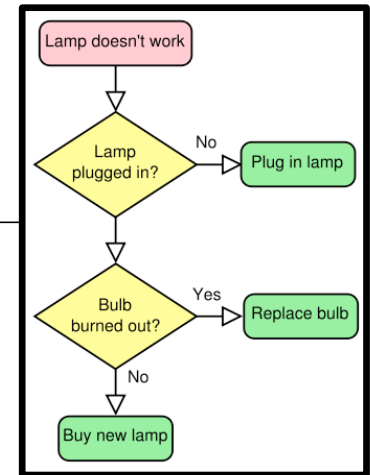
Programming: Choice of an Algorithm

- **Goal:**

- solution of the problem
- independent of programming language
- logic

- **Tools** (similar in most programming languages but sometimes with different names):

- | | |
|-------------------------|-------------------|
| - Assignment/initialize | - Loop |
| - Call | - OPEN/CLOSE file |
| - IF...THEN...ELSE | - READ |
| - Increment | - WRITE |
| - Module/function | |



There are several ways to solve a programming task! Even small changes in the specification may require a new algorithm!

Programming: Choice of an Algorithm

- **Goal:**

- solution of the problem
- independent of programming language
- logic

- **Tools** (similar in most programming languages but often with different names):

- | | |
|-------------------------|-------------------|
| - Assignment/initialize | - Loop |
| - Call | - OPEN/CLOSE file |
| - IF...THEN...ELSE | - READ |
| - Increment | - WRITE |
| - Module | |



xkcd.com

There are several ways to solve a programming task! Even small changes in the specification may require a new algorithm!

Program to manipulate data



Need to store data



Variables

Memory needed to store data



Variables come in **different types!**

A key question is **how much memory** (how many bits) are needed **to store a given variable!**

For a programmer, the glass is neither half full, nor half empty – it is **twice as large as necessary!**

Data Types

- **C is a statically typed language**
 - **The type of a variable is defined before compilation**
 - **A variable has a type and a “name” coupled together**
 - **This identifies a memory location where data can be stored**
 - **Data types are needed to organize data storage and determine which operations are possible ...**
-

Data Types

- *int a;* // **declaration** of variable “a” of type int (integer)
 - *int a=5;* // **declaration and assignment** – gives the variable a value. This is called **initialization**
 - Size (in memory) of the object depends on the type
 - Some basic types in C - there are more ... (next lectures ...)
 - **Integers**
 - **Decimals** // floating point numbers
 - **Characters**
-

Integer Types

bit = 0,1
1 byte = 8 bit

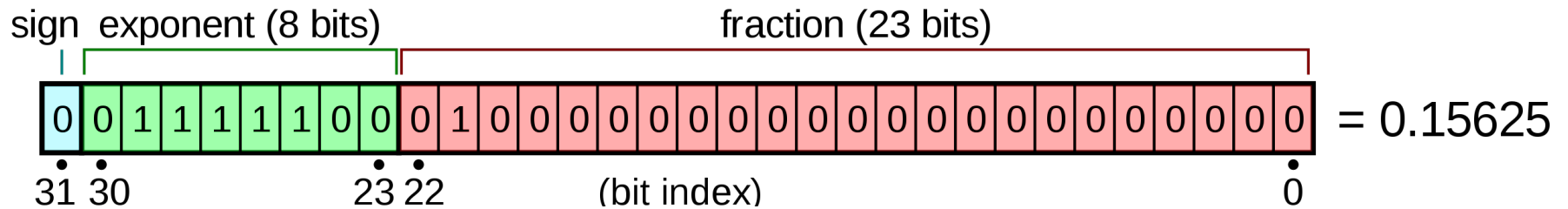
- **Integers**
 - **short int** -> min size: 2 bytes
 - **int** -> min size: 2 bytes
 - **long int** -> min size: 4 bytes
 - **long long int** -> min size: 8 bytes
 - **unsigned or signed** (default, 1 bit for the sign)
 - **char** (character) -> 1 byte
 - **Memory size may depend on machine architecture and compiler**
 - **Use the right size, don't waste memory, know your platform/compiler.**
-

Floating Point Numbers

- Examples: **114.15**, **-0.11415E3**, **22323.3f**
 - sign, fraction and exponent
 - **rounding** -> finite precision
 - float, double and long double
-

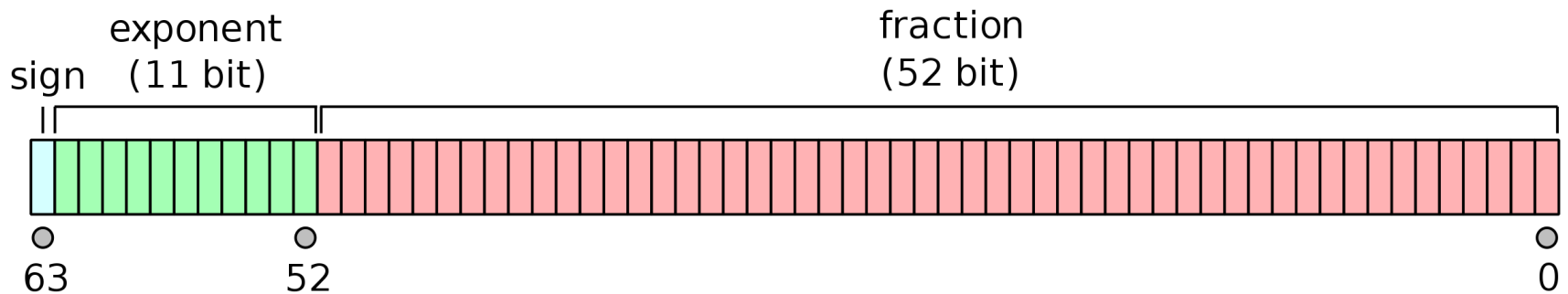
Floating Point Numbers

- **float (32 bits)**



Floating Point Numbers

- **double (64 bits)**



- **Long double (80 bits to 128 bits, typically 96 bits)**

Type Conversion (casting)

- automatic (implicit) or “manual” (explicit) “**casting**”
- automatic:

```
int a;  
double b=3.7;  
a = b; // assignment: b is casted to type int ,  
           // a becomes 3  
float d, c = 5.3;  
d = c * a; // operand with the highest type  
           // determines type of the right-  
           // hand side; d becomes 15.9
```

- explicit: use **typecast** - (type) expression // use carefully!

Example:

```
int e, g;  
e = (int) b; // e is of type int and has value 3  
long double f = 4.18478E14;  
g = (int) f; // (int)f is not defined!
```

See chapter 3.6!

Example

```
/* Calculate the worth of your weight in platinum */
#include <stdio.h>
int main(void) {
    float weight; /* user weight */
    float value; /* platinum equivalent */

    printf("Your worth in platinum?\n");
    printf("Let's check it out.\n");
    printf("Please enter your weight in kg: ");

    /* get input from the user */
    scanf("%f", &weight);
    /* assume platinum price is 8000 SEK per ounce; 32.1507466 converts kg to ounces (troy) */
    value = 8000.0f * weight * 32.1507466f; /* is this correct? */
    printf("Your weight in platinum is worth $%.2f.\n", value); /* %.2f is a conversion specifier */

    return 0; }
```

Characters

- **ASCII characters**
 - 7 bits
 - 128 characters
 - A-Z, a-z, control sequences, and symbols
 - åäö may be found in LATIN-1 (256 characters based on ASCII)
 - Treated by C as small integers
-

ASCII Table

Dec	Hx	Char	Dec	Hx	HTML	Char	Dec	Hx	HTML	Char	Dec	Hx	HTML	Char
0	0	NUL (null)	32	20	 	Space	64	40	@	@	96	60	`	`
1	1	SOH (Start of heading)	33	21	!	!	65	41	A	A	97	61	a	a
2	2	STX (Start of text)	34	22	"	"	66	42	B	B	98	62	b	b
3	3	ETX (End of text)	35	23	#	#	67	43	C	C	99	63	c	c
4	4	EOT (End of transmission)	36	24	$	\$	68	44	D	D	100	64	d	d
5	5	ENQ (Enquiry)	37	25	%	%	69	45	E	E	101	65	e	e
6	6	ACK (Acknowledge)	38	26	&	&	70	46	F	F	102	66	f	f
7	7	BEL (Bell)	39	27	'	'	71	47	G	G	103	67	g	g
8	8	BS (Backspace)	40	28	(	(	72	48	H	H	104	68	h	h
9	9	TAB (Horizontal tab)	41	29	)	)	73	49	I	I	105	69	i	i
10	A	LF (NL line fd, new line)	42	2A	*	*	74	4A	J	J	106	6A	j	j
11	B	VT (Vertical tab)	43	2B	+	+	75	4B	K	K	107	6B	k	k
12	C	FF (NP form fd, new page)	44	2C	,	,	76	4C	L	L	108	6C	l	l
13	D	CR (Carriage return)	45	2D	-	-	77	4D	M	M	109	6D	m	m
14	E	SO (Shift out)	46	2E	.	.	78	4E	N	N	110	6E	n	n
15	F	SI (Shift in)	47	2F	/	/	79	4F	O	O	111	6F	o	o
16	10	DLE (Data link escape)	48	30	0	0	80	50	P	P	112	70	p	p
17	11	DC1 (Device control 1)	49	31	1	1	81	51	Q	Q	113	71	q	q
18	12	DC2 (Device control 2)	50	32	2	2	82	52	R	R	114	72	r	r
19	13	DC3 (Device control 3)	51	33	3	3	83	53	S	S	115	73	s	s
20	14	DC4 (Device control 4)	52	34	4	4	84	54	T	T	116	74	t	t
21	15	NAK (Negative acknowledge)	53	35	5	5	85	55	U	U	117	75	u	u
22	16	SYN (Synchronous idle)	54	36	6	6	86	56	V	V	118	76	v	v
23	17	ETB (End of trans. block)	55	37	7	7	87	57	W	W	119	77	w	w
24	18	CAN (Cancel)	56	38	8	8	88	58	X	X	120	78	x	x
25	19	EM (End of medium)	57	39	9	9	89	59	Y	Y	121	79	y	y
26	1A	SUB (Substitute)	58	3A	:	:	90	5A	Z	Z	122	7A	z	z
27	1B	ESC (Escape)	59	3B	;	;	91	5B	[	[	123	7B	{	{
28	1C	FS (File separator)	60	3C	<	<	92	5C	\	\	124	7C	|	
29	1D	GS (Group separator)	61	3D	=	=	93	5D	]	]	125	7D	}	}
30	1E	RS (Record separator)	62	3E	>	>	94	5E	^	^	126	7E	~	~
31	1F	US (Unit separator)	63	3F	?	?	95	5F	_	_	127	7F		DEL

Char and int – an example

```
#include <stdio.h>
#include <stdlib.h> // needed for the function system
int main(void)
{
    int i = 17;
    char c = 'c'; // ascii value is 99
    int valuec;
    valuec=(int)c;
    int sum;

    sum = i + valuec; // not possible in most languages
    printf("Value of sum : %d\n", sum );
    system("cmd /C pause"); // This works only in Windows
    return 0;
}
```

Enumerated Type (enum)

- **User-defined data type**
 - **Assign names to integral constants (which makes the program easy to read and maintain...)**
 - **declared in the header file `stdio.h`**
 - **Possible use: a group of constants, flags, etc.**
 - **Example:**

```
enum week{Mon, Tue, Wed, Thu, Fri, Sat, Sun};  
// automatically Mon = 0, Tue = 1, Wed = 2, etc.  
enum const{a=1, b=23, c, d=41};  
//a=1, b=23, c=24, d=41
```
-

Example

```
#include <stdio.h>

enum week{mon,tue,wed,thu,fri,sat,sun};

int main()
{
    // create the variable 'day' of enum 'week' type
    enum week day;
    day=wed; // 'day' takes the value of 'wed'
    printf("day= %d\n\n",day); //print the value 'day'

    //exit the program
    printf("Press enter to exit...");
    getchar();

    return 0;
}
```

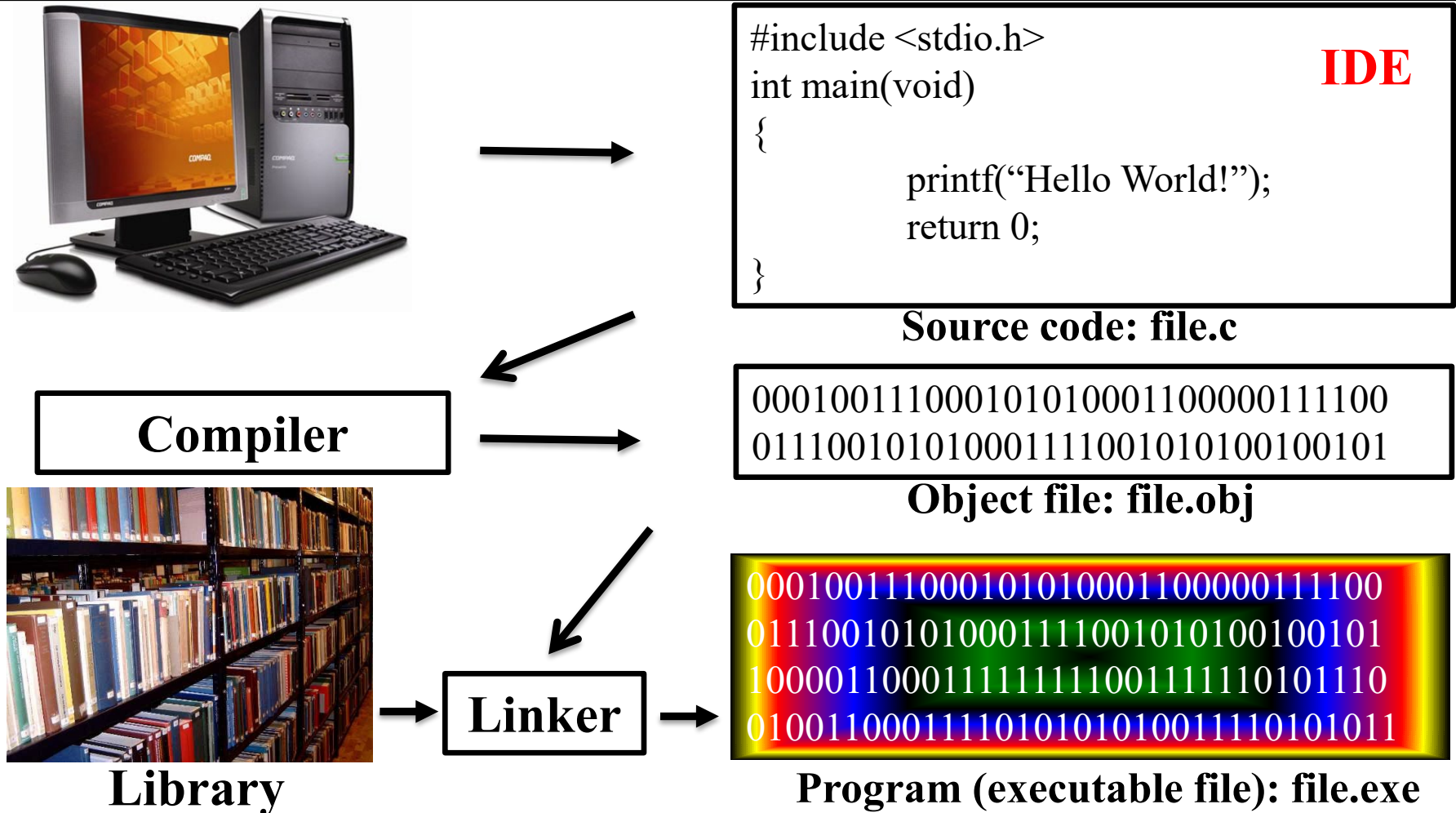
Void



- **void** = "**empty**" data type, no value
 - **Examples**
 - *void myfunction(...);* // the function returns no value
 - *int function_name(void);* // the function does not accept arguments
 - more on type void in connection with **pointers** and functions in next lectures
-

Preprocessor

How to Create a Program?



Preprocessor

file.c → **Preprocessor** → file.c* → **Compiler** → file.obj

```
/* Conversion Fahrenheit -> Celsius */
#include <stdio.h>
#define FREEZING_PT 32.0f
#define SCALE_FACTOR (5.0f / 9.0f)

int main (void)
{
    float fahrenheit, celsius;
    printf ("Enter Fahrenheit: ");
    scanf ("%f", &fahrenheit);
    celsius = (fahrenheit - FREEZING_PT) *
SCALE_FACTOR;
    printf("Celsius: %.1f\n", celsius);
    return 0;
}
```

```
blank line
lines brought in from stdio.h
blank line
blank line
blank line
int main (void)
{
    float fahrenheit, celsius;
    printf ("Enter Fahrenheit: ");
    scanf ("%f", &fahrenheit);
    celsius = (fahrenheit - 32.0f) * (5.0f
/9.0f);
    printf("Celsius: %.1f\n", celsius);
    return 0;
}
```

Preprocessor

- includes **header files** (.h)
 - provides access to functions from standard libraries
 - allows more **structured programming**
 - header files available in the IDE, **<name.h>**
 - header files written by the user (see chapter 9.3), **"name.h"**
 - defines **MACRO**...
 - ... and more ... see, e.g., conditional compiling
 - Chapter 9.4 in VtC
-

Preprocessor: macro

```
#define BASE 50 // something that rarely changes but has no type!  
#include <stdio.h>
```

MACRO BASE initialized with value 50

```
int main (void)  
{  
    float number, price;  
    printf ("Number of kWh?\n"); // can change at any time  
    scanf ("%f", &number);  
    printf ("Price of kWh?\n"); // also this can change  
    scanf ("%f", &price);  
    printf ("Bill: %.2f", (number * price) + BASE);  
    return 0;  
}
```

const vs. macro

```
/*# define BASE 50 */ // changes rarely; easy to edit
#include <stdio.h>

int main (void)
{
    const int base = 50; // constant of type int instead of macro
    float number, price;
    printf ("Number of kWh?\n"); // can change at any time
    scanf ("%f", &number);
    printf ("Price of kWh?\n"); // also this can change
    scanf ("%f", &price);
    printf ("Bill: %.2f", (number * price) + base);
    return 0;
}
```

Summary of Lecture 2

- **Vocabulary – we need to speak properly ...**
 - **Algorithms – how we organize the instructions needed for the solution of a problem ...**
 - **Data types – integers, floats, chars – the way we store data ...**
 - **Preprocessor**
-